

PROJECTFUEL

Red de distribución de combustible

DIAGRAMA ENTIDAD-RELACIÓN

Modelo de datos — ProjectFuel Backend

Universidad Piloto de Colombia

Autor: Juan Diego Galindo

Repositorio: github.com/juandiegogalindo/ProjectFuel-Backend

Tabla de contenido

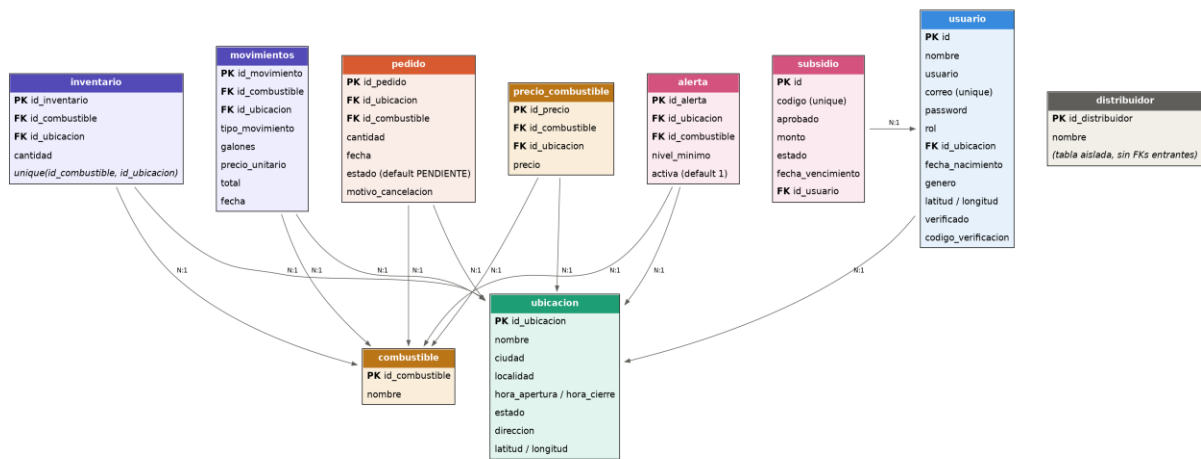
Tabla de contenido.....	2
1. Introducción.....	3
2. Diagrama	3
3. Descripción de entidades	3
3.1 usuario	3
3.2 ubicacion.....	4
3.3 combustible.....	4
3.4 inventario	4
3.5 movimientos	4
3.6 pedido	4
3.7 precio_combustible.....	5
3.8 alerta	5
3.9 subsidio	5
3.10 distribuidor.....	5
4. Relaciones.....	5
5. Hallazgos y limitaciones del modelo	6

1. Introducción

Este documento describe el modelo de datos de ProjectFuel, correspondiente a las diez entidades JPA definidas en el backend (paquete `projectfuel.backend.entity`) y persistidas sobre una base de datos MySQL. El esquema es generado y actualizado automáticamente por Hibernate mediante la propiedad `spring.jpa.hibernate.ddl-auto=update`, por lo que no existen actualmente scripts de migración SQL versionados de forma independiente al código.

El diagrama fue construido a partir de la revisión directa de las clases de entidad y sus anotaciones `@ManyToOne` / `@JoinColumn`, por lo que refleja fielmente las relaciones tal como existen en el código, incluyendo un caso de tabla aislada que se documenta en la sección de hallazgos.

2. Diagrama



3. Descripción de entidades

3.1 usuario

Campo	Tipo	Descripción
PK id	Integer	Identificador autogenerado del usuario
nombre	String	Nombre completo
usuario	String	Nombre de usuario, único a nivel de aplicación
correo	String	Correo electrónico, restricción unique en base de datos
password	String	Contraseña almacenada en texto plano (ver Limitaciones)
rol	String	cliente, operador, distribuidor o admin
FK id_ubicacion	Integer	Estación de referencia del usuario (nullable)
fecha_nacimiento, genero	String	Datos demográficos opcionales
latitud, longitud	Double	Última posición conocida del usuario
verificado	Integer	Bandera de verificación de cuenta (default 0)

codigo_verificacion	String	Código asociado al proceso de verificación
---------------------	--------	--

3.2 ubicacion

Campo	Tipo	Descripción
PK id_ubicacion	Integer	Identificador autogenerado de la estación
nombre, direccion	String	Identificación y dirección física de la estación
ciudad, localidad	String	Agrupación geográfica usada para precios por zona
hora_apertura, hora_cierre	String	Horario de atención
estado	String	Estado operativo de la estación
latitud, longitud	Double	Coordenadas usadas por el mapa del frontend

3.3 combustible

Campo	Tipo	Descripción
PK id_combustible	Integer	Identificador autogenerado
nombre	String	Tipo de combustible (por ejemplo, corriente, extra, ACPM)

3.4 inventario

Campo	Tipo	Descripción
PK id_inventario	Integer	Identificador autogenerado
FK id_combustible	Integer	Combustible almacenado
FK id_ubicacion	Integer	Estación donde se almacena
cantidad	Double	Galones disponibles
—	—	Restricción unique(id_combustible, id_ubicacion): una fila por combinación

3.5 movimientos

Campo	Tipo	Descripción
PK id_movimiento	Integer	Identificador autogenerado
FK id_combustible / id_ubicacion	Integer	Combustible y estación afectados
tipo_movimiento	String	ENTRADA o SALIDA
galones	Double	Cantidad movida
precio_unitario, total	Double	Valor unitario y total calculado del movimiento
fecha	String	Fecha del movimiento (almacenada como texto, no como tipo fecha nativo)

3.6 pedido

Campo	Tipo	Descripción
PK id_pedido	Integer	Identificador autogenerado
FK id_ubicacion / id_combustible	Integer	Estación solicitante y combustible pedido
cantidad	Double	Galones solicitados
fecha	String	Fecha programada de entrega

estado	String	PENDIENTE (default), ACEPTADO, ENTREGADO o CANCELADO
motivo_cancelacion	String	Motivo registrado si el pedido fue cancelado

3.7 precio_combustible

Campo	Tipo	Descripción
PK id_precio	Integer	Identificador autogenerado
FK id_combustible / id_ubicacion	Integer	Combinación combustible + estación a la que aplica el precio
precio	Double	Precio vigente

3.8 alerta

Campo	Tipo	Descripción
PK id_alerta	Integer	Identificador autogenerado
FK id_ubicacion / id_combustible	Integer	Estación y combustible monitoreados
nivel_minimo	Double	Umbral que dispara la alerta de inventario bajo
activa	Integer	Bandera de activación (default 1)

3.9 subsidio

Campo	Tipo	Descripción
PK id	Integer	Identificador autogenerado
codigo	String	Código del subsidio, restricción unique
aprobado	Integer	Bandera de aprobación
monto	Double	Monto cubierto por el subsidio
estado	String	Estado del subsidio
fecha_vencimiento	String	Fecha límite de validez
FK id_usuario	Integer	Usuario titular del subsidio

3.10 distribuidor

Campo	Tipo	Descripción
PK id_distribuidor	Integer	Identificador autogenerado
nombre	String	Nombre del distribuidor

4. Relaciones

- usuario N:1 ubicacion — un usuario tiene, opcionalmente, una estación de referencia.
- subsidio N:1 usuario — un subsidio pertenece a exactamente un usuario titular.
- inventario N:1 combustible, inventario N:1 ubicacion — con restricción unique conjunta, de modo que cada estación tiene como máximo una fila de inventario por tipo de combustible.
- movimientos N:1 combustible, movimientos N:1 ubicacion — cada movimiento de entrada o salida queda asociado a una estación y un combustible.

- pedido N:1 ubicacion, pedido N:1 combustible — cada pedido es solicitado por una estación para un combustible específico.
- precio_combustible N:1 combustible, precio_combustible N:1 ubicacion — el precio se define por combinación de combustible y estación.
- alerta N:1 ubicacion, alerta N:1 combustible — cada alerta de inventario mínimo aplica a una combinación específica.

5. Hallazgos y limitaciones del modelo

Documentados aquí por transparencia, a partir de la revisión directa de las entidades:

- Tabla distribuidor aislada: la entidad DistribuidorEntity no tiene ninguna relación entrante ni saliente con el resto del modelo. Ningún pedido, movimiento o usuario referencia una fila de esta tabla. En la práctica, el rol "distribuidor" en la aplicación se maneja completamente a través del campo rol de usuario, y la tabla distribuidor parece ser un vestigio no integrado al flujo real de negocio.
- Fechas almacenadas como texto: los campos de fecha (fecha en movimientos y pedido, fecha_nacimiento en usuario, fecha_vencimiento en subsidio) están tipados como String en lugar de un tipo de fecha nativo (LocalDate o Date), lo cual no garantiza un formato consistente ni permite comparaciones o rangos de fecha de forma nativa en la base de datos.
- Sin esquema de migraciones versionado: al usar ddl-auto=update, el esquema se genera y ajusta automáticamente en cada arranque de la aplicación a partir de las entidades. Esto es adecuado para desarrollo, pero no es una práctica recomendada para producción, donde se esperaría un historial de migraciones explícito (por ejemplo, con Flyway o Liquibase).
- Sin integridad referencial declarada a nivel de negocio: las relaciones @ManyToOne no declaran una política explícita de borrado en cascada, por lo que el comportamiento ante la eliminación de una ubicación o combustible referenciado dependerá del comportamiento por defecto de MySQL/Hibernate y debe verificarse antes de un despliegue real.